

Plano de Execução

Projeto **analise_acoes** — Sistema de projeção do próximo pregão para PETR4, VALE3 e ITUB4 na B3.

Repositório	github.com/vanderbarbosa/analise_acoes
Stack	Python 3.11 · Flask 3 · SQLAlchemy 2 · SQLite · yfinance · feedparser · Plotly · APScheduler
Tickers	PETR4 · VALE3 · ITUB4 (B3, sufixo .SA)
Documento gerado em	2026-05-03
Autor	Vanderlei Barbosa (com apoio do Claude Code)

1. Sumário executivo

O `analise_acoes` é um site Python de apoio à decisão para três ações da B3. Combina **indicadores técnicos** calculados sobre o histórico do yfinance com um **score de notícias** obtido por coleta RSS e classificação heurística, e gera uma **projeção direcional** (alta/baixa/neutro) com magnitude estimada por ATR para o próximo pregão.

A arquitetura, o pipeline de dados e o site Flask já estão implementados e funcionando. Este documento descreve o plano para **concluir o produto** — fechando lacunas de testes, atualização de dados, automação, validação empírica (backtest) e melhorias de UX — em **seis fases** incrementais, organizadas por valor agregado e ordem de dependência. Cada fase é entregável de forma independente e o projeto pode parar entre fases sem deixar estado quebrado.

Esforço total estimado: **~14 a 22 horas** de trabalho (excluindo Fase 6 opcional, +20 h se ativada).

Disclaimer: o produto é estudo / apoio à decisão, não recomendação de investimento. O teto realista de acerto direcional intradiário/diário em um modelo heurístico desta natureza é estimado em 52–58%.

2. Visão do produto

2.1 Propósito

Disponibilizar, em uma interface web simples, uma leitura diária consolidada para PETR4, VALE3 e ITUB4 que ajude a formar opinião sobre a tendência de curto prazo, evitando que o usuário precise consultar múltiplas fontes de cotação, indicadores e notícias.

2.2 Escopo

- Acompanhamento contínuo dos 3 tickers acima (mais podem ser adicionados via `config.py`).
- Coleta diária de notícias via RSS (InfoMoney, Money Times, G1 Economia).
- Classificação heurística por categoria (macroeconômica, política, fundamentos, outro) e sentimento (positivo/negativo).
- Cálculo de indicadores técnicos: RSI(14), MACD(12,26,9), EMAs(9,21,50), Bandas de Bollinger(20,2 σ), ATR(14).
- Projeção do próximo pregão: direção, magnitude estimada (% sobre ATR) e nível de confiança.
- Persistência completa em SQLite para auditoria e backtest.
- Site Flask com listagem de tickers, página de detalhe, gráfico interativo (Plotly) e lista de notícias recentes.

2.3 Fora do escopo

- Execução de ordens, integração com corretora ou qualquer ação automatizada de trading.
- Recomendação personalizada de carteira ou perfil de investidor.
- Análise intradia (timeframes < 1 dia).
- Cobertura de ativos fora dos 3 tickers configurados (sem impedimento técnico, mas fora do MVP).
- Modelos pesados de NLP / Deep Learning na fase inicial — substituíveis na Fase 6 opcional.

3. Estado atual (baseline)

Snapshot tirado em **2026-05-03** a partir do banco `data/analise.db` e da árvore de código do repositório.

3.1 Componentes implementados

Camada de dados	<code>src/db/models.py</code> — schema SQLAlchemy 2.0 com 5 tabelas: <code>tickers</code> , <code>price_daily</code> , <code>news_raw</code> , <code>news_scored</code> , <code>predictions</code>
Pipeline de notícias	<code>src/news/{collect,filter,classify,score}.py</code> — coleta RSS, filtragem por regex+credibilidade, classificação por <code>tfidf</code>
Pipeline de preços	<code>src/prices/{fetch,indicators,signal}.py</code> — download via <code>yfinance</code> , cálculo puro-pandas dos indicadores
Projektor	<code>src/prediction/projector.py</code> — combina <code>news</code> e <code>tech</code> via <code>WEIGHT_NEWS/WEIGHT_TECHNICAL</code>
Jobs	<code>src/jobs/{backfill,daily}.py</code> — backfill histórico do Q1 2026 e job diário (notícias + preços + projeções)
Flask app	<code>app.py</code> — rotas <code>/</code> e <code>/ticker/<symbol></code> , três filtros Jinja (<code>brl</code> , <code>pct</code> , <code>d</code>), gráfico Plotly server-rendered.
Templates / estilo	<code>templates/{base,index,ticker}.html</code> , <code>static/css/app.css</code> (3,5 KB).
DevOps local	<code>.claude/hooks/auto-commit.ps1</code> e <code>.claude/settings.json</code> — auto-commit + push após cada turno de trabalho

3.2 Métricas atuais do banco

Tickers cadastrados	3 (PETR4, VALE3, ITUB4)
Candles em <code>price_daily</code>	183 (2026-01-02 a 2026-03-31)
Notícias brutas (<code>news_raw</code>)	6
Notícias scoradas (<code>news_scored</code>)	6
Predictions geradas	3 (uma por ticker)
Tamanho do arquivo <code>SQLite</code>	127 KB

4. Lacunas identificadas

4.1 Críticas (bloqueiam encerramento do projeto)

- bullet **Defasagem de preços:** backfill termina em 2026-03-31 e a data atual é 2026-05-03 — ~5 semanas de candles ausentes; o site exibe dados desatualizados.
- bullet **Suite de testes inexistente:** a pasta tests/ contém apenas `__init__.py`. Não há cobertura para indicadores, sinais técnicos, classificador, agregador de notícias ou projetor — riscos altos de regressão silenciosa.
- bullet **Sem validação empírica:** não há backtest comparando `predictions.for_date` com o close real. Sem isso, a tese de “52–58 % de acerto” permanece não verificada.
- bullet **Pipeline manual:** `daily.py` precisa ser executado à mão. Falta scheduler (APScheduler já está em `requirements.txt` mas não é usado).

4.2 Importantes (qualidade do produto)

- bullet **Logging estruturado ausente** — usa apenas `print()`. Dificulta diagnóstico em execução automatizada.
- bullet **Tratamento de erros em yfinance:** rede caída, ticker inválido ou rate-limit não geram `retry/backoff`.
- bullet **RSS pode duplicar artigos** com URL canônica diferente (`utm_source`, `fbclid`). Deduplicação atual é por link estrito.
- bullet **Página de detalhe não mostra histórico** do `tech_score` nem do `news_score` ao longo do tempo.
- bullet **Não há indicador visual** de “última atualização do pipeline” no site, dificultando saber se o número exibido é fresco.

4.3 Desejáveis (evolução pós-MVP)

- bullet **Substituir o classificador heurístico** por NLP real (zero-shot + sentimento PT-BR) — Fase 6 opcional.
- bullet **Adicionar modo as-of <data>** para reprocessar uma data passada (útil para depuração).
- bullet **Permitir cadastro de novos tickers** via UI (hoje só via `config.py`).
- bullet **Exportação CSV** de `predictions` vs realizado.
- bullet **Alerta por e-mail** quando a confiança ultrapassar um limiar.

5. Plano em fases

Cada fase é um entregável fechado. As Fases 1 a 4 são suficientes para considerar o MVP concluído; a Fase 5 polariza melhor a percepção de qualidade; a Fase 6 é opcional e introduz dependência pesada (PyTorch).

Fase 1 — Atualização de dados e estabilização do baseline

Objetivo	Eliminar a defasagem de preços, garantir que o pipeline diário roda end-to-end sem erro, e deixar o sistema pronto para a Fase 2.
Esforço estimado	1,5 a 2,5 h
Pré-requisitos	Nenhum (ponto de partida)
Critério de aceitação	Em data/analise.db existe price_daily até o último pregão útil; executar <code>`python -m src.jobs.daily`</code> sem erros.

Escopo de trabalho

- Atualizar BACKFILL_END em config.py para o último pregão útil disponível.
- Rodar src.jobs.backfill para preencher o gap (estratégia: ampliar janela; o DELETE+INSERT do backfill é idempotente).
- Executar src.jobs.daily uma vez para validar a integração ponta-a-ponta.
- Conferir visualmente as três páginas (/ , /ticker/PETR4, /ticker/VALE3, /ticker/ITUB4).
- Capturar quaisquer erros encontrados (encoding, rate-limit, schema) e abrir tickets internos.

Arquivos afetados

- config.py (BACKFILL_END)
- Possivelmente src/prices/fetch.py (caso surja erro de schema do yfinance)
- data/analise.db (estado atualizado, não versionado)

Riscos específicos

- yfinance pode retornar DataFrame com MultiIndex inesperado em janelas grandes — já tratado em fetch.py mas vale revalidar.
- RSS retorna apenas notícias dos últimos dias; news_raw continuará com volume baixo até a Fase 4 entrar em operação contínua.

Fase 2 — Suite de testes automatizados

Objetivo	Cobrir as funções puras do pipeline com testes determinísticos para permitir refatorações futuras.
Esforço estimado	3 a 5 h
Pré-requisitos	Fase 1 concluída (baseline estável)
Critério de aceitação	<code>`pytest`</code> finaliza com ≥ 25 testes passando em < 5 s, sem chamadas de rede; cobertura mínima de 80%.

Estratégia de teste

bullet Testar apenas funções puras (sem yfinance/RSS/HTTP). Para módulos com I/O, isolar a parte determinística e testar separadamente.

bullet Usar fixtures pytest em conftest.py para criar DataFrames OHLCV sintéticos reproduzíveis (seed fixa).

bullet Para a camada de DB, usar SQLite em :memory: via fixture session (não tocar data/analise.db).

bullet Sem mocks de bibliotecas externas no MVP — preferir testar camadas puras.

Arquivos a criar

bullet tests/conftest.py — fixtures (sample_ohlc, in_memory_session, seeded_tickers).

bullet tests/test_indicators.py — RSI extremo, MACD cruzando zero, Bollinger com vol baixa/alta, ATR consistente com TR manual.

bullet tests/test_signal.py — entradas com NaN, RSI sobrevendido/sobrecomprado, score clipado em [-1, 1].

bullet tests/test_classify.py — categoria por palavra-chave, sentimento positivo/negativo/neutro, normalização de acentos.

bullet tests/test_filter.py — detecção por regex (PETR4, VALE3, ITUB4) sem falsos positivos óbvios.

bullet tests/test_score.py — agregação ponderada em janela, ticker inexistente, conjunto vazio.

bullet tests/test_projector.py — direção em torno da zona morta $\pm 0,15$, magnitude $\times$ confiança, neutro reduz magnitude para 30 %.

bullet tests/test_models.py — init_db idempotente, unique constraints (ticker_id+trade_date, news_id+ticker_id, ticker_id+for_date).

Critérios de qualidade

bullet Sem dependência de rede (pytest --offline equivalente).

bullet Tempo total < 5 s.

bullet Cada teste expressa uma asserção clara em seu nome (test_rsi_oversold_below_30, etc.).

Fase 3 — Backtest e métricas de acurácia

Objetivo	Quantificar empiricamente a qualidade da projeção comparando o que o sistema previu com o que realmente aconteceu.
Esforço estimado	3 a 5 h
Pré-requisitos	Fases 1 e 2 concluídas (dados atualizados e código testado)
Critério de aceitação	Comando <code>`python -m src.jobs.backtest`</code> produz, no console e em <code>data/backtest_report.json</code> , um relatório de backtest.

Escopo de trabalho

- Criar `src/jobs/backtest.py` que percorre todas as predictions e busca o close do `for_date` no `price_daily`.
- Calcular: direção realizada (sinal de `close[for_date] - close[generated_at_date]`), magnitude realizada (% return), MAE da magnitude prevista, taxa de acerto direcional global e por ticker.
- Exibir métricas estratificadas por nível de confiança (bins 0-0.25, 0.25-0.5, 0.5-0.75, 0.75-1.0).
- Criar rota Flask `/backtest` e template `templates/backtest.html` com tabela.
- Persistir snapshot do relatório em `data/backtest_report.json`.

Métricas a reportar

- Acerto direcional global (% das vezes em que direção prevista == realizada, ignorando neutros).
- Acerto direcional por ticker.
- Acerto condicional: "Quando o sistema diz alta com confiança $\geq X$, qual % das vezes acerta?".
- MAE da magnitude (em pontos percentuais).
- Distribuição de previsões por categoria (alta/baixa/neutro) — sanity check de viés.
- Comparação com baseline ingênuo ("sempre alta" e "última direção").

Arquivos a criar/modificar

- `src/jobs/backtest.py` (novo)
- `tests/test_backtest.py` (novo)
- `app.py` (adicionar rota `/backtest`)
- `templates/backtest.html` (novo)

Fase 4 — Scheduler e operação contínua

Objetivo	Eliminar a operação manual: o pipeline diário deve rodar sozinho em horário compatível com o horário de mercado.
Esforço estimado	2 a 3 h
Pré-requisitos	Fase 3 concluída (não obrigatório, mas evita publicar métricas vazias)
Critério de aceitação	Subir o servidor com <code>`python app.py`</code> agenda automaticamente: (a) <code>`daily.run()`</code> todos os dias úteis.

Escopo de trabalho

- Criar `src/jobs/scheduler.py` com `BackgroundScheduler` do `APScheduler`.

- Definir cron jobs: `daily.run()` em dias úteis (Mon-Fri) 19:30, e `ingest_news()` ao meio-dia.
- Persistir último `run_at` e último status (success/failed) em uma nova tabela ou em `data/runs.json`.
- Iniciar o scheduler dentro de `app.py` somente quando `__name__ == '__main__'` (evitar duplicação no debug-reload).
- Adicionar componente visual no topo do site: 'Atualizado em ' + indicador colorido.
- Logar para arquivo `data/logs/scheduler.log` com rotação simples.

Considerações operacionais

- AP Scheduler em modo Background sobrevive a requests do Flask, mas morre se o processo for encerrado — para servidor 24x7, considerar `systemd`/Task Scheduler do Windows.
- Documentar no `CLAUDE.md` como iniciar e parar o serviço.
- Adicionar variável de ambiente `SCHEDULER_DISABLED=1` para rodar o site sem ativar jobs (útil em dev).

Arquivos a criar/modificar

- `src/jobs/scheduler.py` (novo)
- `src/db/models.py` (talvez nova tabela `run_log`)
- `app.py` (instanciar scheduler, exibir status)
- `templates/base.html` (exibir badge de status)
- `static/css/app.css` (estilos do badge)

Fase 5 — Melhorias de UI e observabilidade

Objetivo	Refinar a experiência do usuário e aumentar a transparência do que o sistema está fazendo por
Esforço estimado	3 a 4 h
Pré-requisitos	Fases 1-4 concluídas
Critério de aceitação	Usuário consegue, sem instruções, entender (a) por que uma projeção foi alta/baixa/neutra olha

Itens de trabalho

- Gráfico secundário em /ticker/<symbol>: linha temporal do tech_score e bar chart do news_score diário (últimos 60 pregões).
- Tooltip explicativo em cada notícia: "contribuiu +0,12 ao news_score (categoria fundamentos × peso 1,0 × relevância 0,7 × sentimento +0,17)".
- Página /sobre explicando metodologia, fontes e disclaimer (link no footer).
- Migrar prints para logging estruturado (logging.getLogger(__name__)) com níveis INFO/WARNING/ERROR.
- Adicionar tratamento de exceções no daily.py para que falha em um ticker não derrube os outros.
- Deduplicação de URL canonicalizada no filter (remover utm_*, fbclid, ?ref=).

Arquivos afetados

- templates/ticker.html (gráficos extras + tooltips)
- templates/sobre.html (novo)
- app.py (rota /sobre, helpers de gráfico)
- src/news/filter.py (canonicalização de URL)
- src/jobs/daily.py (try/except por ticker)
- Vários src/**/*.py (substituir print por logging)

Fase 6 — NLP real para classificação de notícias [opcional]

Objetivo	Substituir o classificador heurístico por modelos pré-treinados de linguagem em português, mel
Esforço estimado	5 a 8 h (mais ~1,5 GB de download de modelos)
Pré-requisitos	Fases 1-5 concluídas e baseline de acurácia coletado (Fase 3) para comparação A/B
Critério de aceitação	Em pipeline paralelo (não substitutivo no primeiro momento), o classificador NLP produz score c

Escopo de trabalho

- Descomentar transformers, torch e sentencepiece em requirements.txt.
- Substituir classify_category por zero-shot com modelo PT-BR (ex: 'mDeBERTa-v3-base-mnli-xnli').
- Substituir score_sentiment por modelo de sentimento PT-BR (ex: 'cardiffnlp/twitter-xlm-roberta-base-sentiment').
- Cachear modelos em data/models/ para não baixar a cada run.

- Rodar backtest A/B comparando heurístico vs NLP — decidir promoção com base em métricas, não em fé.

Cuidados

- Pacote final fica grande (PyTorch ~600 MB).

- Inferência custa ~50-200 ms por notícia em CPU; aceitável em job diário, não em request síncrono do Flask.

- Documentar no CLAUDE.md como ativar/desativar via flag em config.py.

6. Riscos e mitigações

Risco	Mitigação
yfinance ficar instável ou bloquear IP	Backfill/daily fazem retry implícito do urllib3; em produção contínua, considerar fallback para Ticker
RSS retornar volume insuficiente de notícias para o score	Classifier não depende só do tech_score (peso 0,45 vira default)
Classificador heurístico ter falsos positivos (ex Vale Tech/B3, BATIFRUS, etc)	Regulate TICKER_BATTERIES (exemplos) no texto ("vale s.a.", "vale3", "mineradora")
Falha em um ticker derrubar o job inteiro	Fase 5 envolve cada ticker em try/except no daily.py.
Auto-commit publicar dados sensíveis ou estragando o repo	gitignore para data/, .env, .venv; CLAUDE.md alerta para não terminar turno
Acurácia ficar abaixo de baseline ingênuo	Backtest da Fase 3 expõe isso explicitamente. Caminhos: ajustar pesos em con

7. Métricas de sucesso do projeto

- Funcional:** 100 % das fases 1-4 concluídas com critérios de aceitação atendidos.
- Qualidade:** ≥ 25 testes passando em < 5 s; cobertura ≥ 80 % nos módulos puros.
- Operacional:** 7 dias consecutivos com pipeline diário rodando sem intervenção manual.
- Métrica de produto (validação):** taxa de acerto direcional ≥ 50 % no backtest (qualquer valor abaixo é aceitável desde que reportado honestamente — o objetivo do projeto é estudo, não rentabilidade garantida).
- Documentação:** CLAUDE.md atualizado refletindo qualquer mudança de comando, e README opcional para visitantes externos do GitHub.

8. Roadmap futuro (pós-MVP)

Itens fora do escopo do plano atual mas naturalmente conectados — ficam como referência para uma eventual v2:

- Adição de novos tickers (BBAS3, WEGE3, MGLU3) — só requer entradas em config.TICKERS e re-backfill.
- Deploy real (Render, Fly.io ou VPS) com worker separado para o scheduler.
- Substituir SQLite por PostgreSQL (2-3 horas de migração com SQLAlchemy).
- Treinamento supervisionado: usar histórico de predictions vs realizado como dataset para um classificador próprio (XGBoost ou regressão logística sobre features técnicas + agregados de notícia).
- Webhook/Telegram bot para enviar projeções pela manhã.
- Integração com fontes de notícias institucionais via API (CVM, B3, releases das próprias empresas).
- Modo "comparar pesos": UI que permite alterar WEIGHT_NEWS e WEIGHT_TECHNICAL e visualizar impacto retroativo.

Apêndice A — Comandos de referência

Todos os comandos abaixo assumem que o cwd é a raiz do projeto e usam o venv local (.venv\Scripts\python.exe). A flag `-X utf8` é obrigatória no Windows para evitar erro de encoding com caracteres como →, ç, ã.

Setup inicial

```
# Instalar dependências
.\.venv\Scripts\python.exe -m pip install -r requirements.txt

# Bootstrap do banco e seed dos tickers
.\.venv\Scripts\python.exe -X utf8 -c "from src.db.models import init_db; init_db()"
```

Backfill histórico

```
.\.venv\Scripts\python.exe -X utf8 -m src.jobs.backfill
```

Pipeline diário

```
.\.venv\Scripts\python.exe -X utf8 -m src.jobs.daily
```

Servidor web

```
.\.venv\Scripts\python.exe -X utf8 app.py
# acesse http://127.0.0.1:5000
```

Testes

```
.\.venv\Scripts\python.exe -m pytest # todos
.\.venv\Scripts\python.exe -m pytest tests/test_X.py::test_Y # único
```

Backtest (Fase 3, ainda a criar)

```
.\.venv\Scripts\python.exe -X utf8 -m src.jobs.backtest
```

Regenerar este PDF

```
.\.venv\Scripts\python.exe -X utf8 scripts\generate_plan_pdf.py
```

Apêndice B — Estrutura de diretórios

```
analise_acoes/
├── app.py                # Flask entrypoint, rotas e filtros Jinja
├── config.py             # Tickers, feeds, regex, pesos, períodos – singl
├── e source of truth
├── requirements.txt      # Dependências (NLP comentado por padrão)
├── CLAUDE.md             # Guia para Claude Code (comandos, convenções, h
├── ooks)
├── PLANO_EXECUCAO.pdf    # Este documento
├── data/
├── ─── analyse.db        # SQLite – schema em src/db/models.py (não vers
├── ionado)
├── ─── models/          # Cache de modelos NLP (Fase 6)
├── ─── scripts/
├── ─── generate_plan_pdf.py # Gera o PDF deste plano
├── ─── src/
├── ─── db/models.py      # SQLAlchemy: Ticker, PriceDaily, NewsRaw, News
├── Scored, Prediction
├── ─── news/{collect,filter,classify,score}.py
├── ─── prices/{fetch,indicators,signal}.py
├── ─── prediction/projector.py
```

```

■     ■■■ jobs/{backfill,daily}.py    # +scheduler.py (Fase 4) +backtest.py (Fase 3)
■■■ templates/
■     ■■■ base.html
■     ■■■ index.html
■     ■■■ ticker.html                # +backtest.html (F3) +sobre.html (F5)
■■■ static/css/app.css
■■■ tests/                           # __init__.py + (Fase 2) suite completa

```

Apêndice C — Fórmulas-chave do projeto

Todas as fórmulas estão implementadas em *pandas puro* (sem *pandas-ta*). Os parâmetros são configuráveis via *config.py*.

RSI (Wilder, período 14)

```

delta = close.diff()
gain = delta.clip(lower=0); loss = -delta.clip(upper=0)
avg_gain = EWM(gain, alpha=1/14); avg_loss = EWM(loss, alpha=1/14)
RSI = 100 - 100 / (1 + avg_gain / avg_loss)

```

MACD (12, 26, 9)

```

macd = EMA(close, 12) - EMA(close, 26)
signal = EMA(macd, 9)
hist = macd - signal

```

Bollinger (20, 2σ)

```

mid = SMA(close, 20)
upper = mid + 2 * STD(close, 20)
lower = mid - 2 * STD(close, 20)

```

ATR (Wilder, 14)

```

TR = max( |high - low|, |high - prev_close|, |low - prev_close| )
ATR = EWM(TR, alpha=1/14)

```

tech_score ∈ [-1, 1]

Soma ponderada de:

- RSI < 30 → +0,5; RSI > 70 → -0,5; senão linear
- MACD hist normalizado por close, peso 0,4
- close vs EMA(21): +0,3 / -0,3
- breakout das bandas: ±0,2

Resultado clipado em [-1, 1].

news_score ∈ [-1, 1]

Para cada notícia da janela de 48 h:

- weight = CATEGORY_WEIGHT[cat] × relevance

$$\text{news_score} = \frac{\sum(\text{sentiment} \times \text{weight})}{\sum(\text{weight})}$$

Projeção combinada

```

combined = news_score * 0,55 + tech_score * 0,45
direction = alta se combined > +0,15; baixa se < -0,15; senão neutro
confidence = min(|combined|, 1)
magnitude = ATR% * (0,4 + 0,6 * confidence)
if neutro: magnitude *= 0,3

```